RoboRacer on a Mac

How the F1TENTH simulator, ROS 2, the ForzaETH race stack and a one-command terminal tool were installed on an Apple Silicon MacBook, and how to use them.

What	Details
Machine	MacBook, Apple Silicon (arm64), macOS 26, 24 GB RAM
Approach	Small Ubuntu 22.04 VMs in OrbStack (no Docker), ROS 2 Humble inside
Disk used	About 8 GB total (4 GB per VM) plus ~150 MB of project files
Control	One command, `rr`, run from the Mac terminal
Viewer	RViz shown in a web browser through noVNC (no accounts, no X11 setup)
Project folder	`~/Documents/Projects/robotics/roboracer`
Set up	September 2026

Why it is built this way

- ROS 2 does not run natively on macOS, so it runs in Linux. The official F1TENTH instructions use Docker; OrbStack VMs were used instead because they are lighter on a Mac, start in seconds, and see the Mac's files directly, so code is edited on the Mac with any editor.
- Two VMs, not one: ForzaETH pins old Python packages (numpy 1.22, scipy 1.7) that break the current official simulator (which needs scipy 1.13+). Keeping them apart means neither can break the other.
- RViz is shown in the browser (noVNC) because it is the one GUI piece, and this avoids XQuartz and Foxglove accounts entirely.
- Algorithms use the same ROS topics as the real car (`/scan` in, `/drive` out), so code tested in the sim runs unchanged on the car.
- Autoware was skipped on purpose: tens of GB, aimed at x86 with NVIDIA GPUs, and not needed for testing racing algorithms.

How the pieces fit

Layer	Runs where	What it is
rr	Mac terminal	Bash script; every command in this guide goes through it
VM "roboracer"	OrbStack	Ubuntu 22.04, ROS 2 Humble (ros-base + RViz), f1tenth_gym + f1tenth_gym_ros (dev-humble branches), your algorithms. RViz at http://localhost:6080
VM "forzaeth"	OrbStack	Clone of the first VM plus ETH Zurich's race stack (ros2-humble). RViz at http://localhost:6081
Real car	Its own Jetson	Keeps its own ROS install. The Mac only connects over SSH

1. Install, step by step

Everything below was run from the Mac terminal. The scripts live in the project's `scripts/` folder and are safe to re-run.

Step 1: OrbStack (the VM manager)

```
brew install --cask orbstack
open -a OrbStack                      # finish its first-run setup
orbctl create ubuntu:jammy roboracer   # Ubuntu 22.04, arm64
```

Step 2: Get the official simulator (Humble branches)

```
mkdir -p ~/Documents/Projects/robotics/roboracer/{ws/src,scripts,notes}
cd ~/Documents/Projects/robotics/roboracer
git clone -b dev-humble https://github.com/f1tenth/f1tenth_gym.git
git clone -b dev-humble https://github.com/f1tenth/f1tenth_gym_ros.git ws/src/f1tenth_gym_ros
```

The `main` branch of f1tenth_gym_ros targets ROS 2 Foxy; `dev-humble` matches Ubuntu 22.04 and Humble.

Step 3: Provision the VM (scripts/vm_setup.sh)

Run inside the VM (the Mac folder shows up at the same path there):

```
orb -m roboracer bash ~/Documents/Projects/robotics/roboracer/scripts/vm_setup.sh
```

What it does, in order:

- Adds the ROS 2 apt repository and installs `ros-humble-ros-base`, `ros-humble-rviz2`, colcon, rosdep and keyboard teleop (not the full desktop install, to stay small).
- Installs a virtual display for RViz: `tigervnc-standalone-server novnc websockify openbox libgl1-mesa-dri`.
- Upgrades pip for the user, then installs f1tenth_gym in editable mode with the constraint `numpy<2` (ROS Humble's Python packages are built against numpy 1.x).
- Runs `rosdep install` for the workspace (nav2 map server, foxglove bridge, ackermann messages, ...) and `colcon build --symlink-install`.
- Adds one line to the VM's ~/.bashrc that sources `scripts/env.sh`.

Step 4: The rr command

```
chmod +x ~/Documents/Projects/robotics/roboracer/rr
ln -s ~/Documents/Projects/robotics/roboracer/rr /opt/homebrew/bin/rr
rr help
```

rr wraps `orb -m <vm> bash -c ...`, keeps the sim, RViz, the crash auto-reset and the VNC display in tmux sessions inside the VM (so they survive closing the terminal), and opens the browser view.

Step 5: Tracks

```
git clone --depth 1 https://github.com/f1tenth/f1tenth_racetracks.git tracks/src
python3 scripts/make_tracks.py           # 23 real F1 circuits -> tracks/<Name>/
python3 scripts/gen_track.py squiggle    # small practice circuit
python3 scripts/gen_track.py random 7    # random competition-style circuit, seed 7
```

The downloaded track images are white with thin outlines, which lets a car wander off. make_tracks.py redraws each track from its centerline and widths (track white, everything else black), so every map is a closed circuit. It

also copies the centerline and raceline CSVs next to the map, which turns on lap counting in the simulator.

Step 6 (optional): ForzaETH, ETH Zurich's race stack

```
orbctl stop roboracer && orbctl clone roboracer forzaeth && orbctl start roboracer forzaeth
mkdir -p forzaeth/ws/src && cd forzaeth/ws/src
git clone -b ros2-humble --recurse-submodules https://github.com/ForzaETH/race_stack.git
orb -m forzaeth bash ~/Documents/Projects/robotics/roboracer/scripts/forza_setup.sh
```

forza_setup.sh follows their manual-install notes and Dockerfile: removes the first VM's Python packages, installs `linux_req.txt` and `python_req.txt`, quadprog, their f1tenth_gym fork and the TUM race-line optimizer, links the workspace to `~/ws` (their launch files expect it) and builds all 28 packages. It uses the Zenoh middleware (`RMW_IMPLEMENTATION=rmw_zenoh_cpp`) like they do.

2. Problems we hit, and the fixes

Useful if you set this up yourself. Every fix is already built into the scripts.

Symptom	Cause	Fix
pip error: "uv-build was not properly installed"	Ubuntu 22.04 ships pip 22, too old for f1tenth_gym's packaging	Upgrade pip for the user first, then install
"AMENT_TRACE_SETUP_FILES: unbound variable"	ROS setup.bash breaks under bash `set -u`	Wrap it: `set +u; source ...; set -u`
Browser stuck at "Connecting..."	A check like `pgrep -f websockify` matched its own command line, so the bridge was never started	Run display services in a tmux session; use the `[w]ebsockify` pattern trick
Page will not load at roboracer.orb.local	That name did not resolve on this Mac	Use http://localhost:6080 (OrbStack forwards VM ports)
Map choice ignored, car drives out of the building	macOS sed has no `\\s`, so the map line was never replaced	Use `[[:space:]]`
Car drives through walls after a crash	The gym only reports collisions, it does not stop the car	`scripts/auto_reset.py` watches `/ego_racecar/collision` and puts the car back on the start line
Gym refuses a map yaml	Its loader rejects extra keys like initial_pose	Separate `forza.yaml` with initial_pose for ForzaETH
ForzaETH breaks the official gym	Conflicting numpy/scipy pins	Separate VM
ForzaETH build: f110_gym and stack_master fail	Gym is pip-only; setup.py globs a folder as a file	COLCON_IGNORE in the gym; filter the globs to files
ForzaETH planner: "No module named global_racetrajectory_optimization"	Needs `gb_opt_setup.sh` from their post-install	pip install -e the optimizer
Mac slows down	ForzaETH's race-line optimizer on a big (450 m) track pegs the CPU for 10+ minutes	Only run it on small tracks, when the Mac is idle

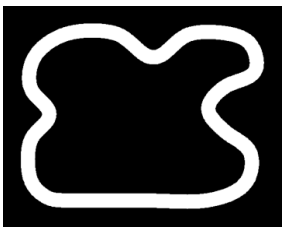
3. Daily use

Command	What it does
`rr sim [track]`	Start the sim and RViz (default Squiggle). Opens http://localhost:6080
`rr run follow_the_gap`	Let an algorithm drive. Ctrl-C stops it and stops the car
`rr ps`	What is running: sim, which algorithm, ROS nodes
`rr reset / rr crashes`	Car back to the start line (crashes do this automatically) / crash count
`rr view follow|chase|top|orbit`	Camera: above the car / behind it / whole track / free 3D
`rr teleop`	Drive with the keyboard (i forward, j and l turn, k stop)
`rr maps`	List every map

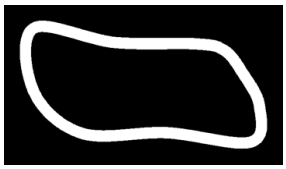
Command	What it does
`rr track new [seed]`	Generate a random competition-style track
`rr new my_idea`	New algorithm from the template
`rr param <algo> <name> <value>`	Tune a running algorithm live
`rr stop`	Stop the sim
`rr down`	Stop the VMs (otherwise they keep running in the background until restart)
`rr forza sim / race / h2h`	ForzaETH sim, time trials on its race line, head-to-head racing
`rr shell, rr logs, rr topics, rr ros ...`	Linux shell, sim logs (tmux), any ros2 command

In RViz (browser): scroll to zoom, drag to rotate or pan, shift+drag to pan. **2D Pose Estimate** then click-drag puts the car somewhere new. The **Views** panel on the right changes the camera by hand.

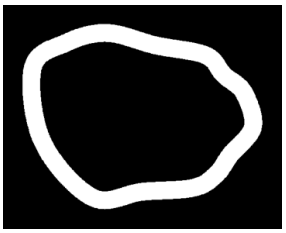
Tracks



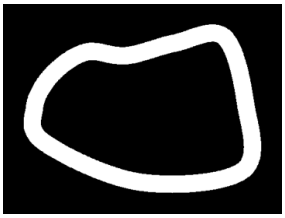
Squiggle



Random1



Random2



Random4

- **Squiggle**: hand-designed 89 m practice loop with a start straight, an S-bend, a 1.0 m hairpin dip and a chicane. The default map.
- **Random<seed>**: generated like procedural racing-game tracks (random points, their outline, edges pushed in and out for hairpins and S-bends, smoothed with a spline), and kept only if it passes F1TENTH-style rules: closed, never crosses itself, walls of neighbouring stretches at least 0.6 m apart, every corner radius at least 1.2 m (the car can turn about 0.75 m), width 1.3 to 1.9 m, lap 35 to 80 m. Same seed gives the same track.
- **23 real F1 circuits** (Monza, Spa, Silverstone, ...) from f1tenth_racetracks, scaled for F1TENTH, 250 to 550 m laps. Good for top speed, less for tight handling.

4. Writing and testing algorithms

Each file in `ws/src/racer_algos/racer_algos/` with a `main()` is an algorithm. Shared plumbing lives in `_base.py`: it subscribes to the lidar and odometry, calls your `on_scan()`, clamps steering to the car's 0.419 rad limit and a max speed, and stops the car on Ctrl-C.

```
from racer_algos._base import RacerNode, spin, deg

class Algo(RacerNode):
    def __init__(self):
        super().__init__('my_idea', speed=1.0, gain=1.0)   # each keyword = live parameter

    def on_scan(self, ranges, angles, scan):
        # numpy arrays; angle 0 = straight ahead, + = left. self.odom = latest odometry
        left, right = self.range_at(ranges, angles, deg(90)), self.range_at(ranges, angles, deg(-90))
        return self.p('speed'), self.p('gain') * (left - right) * 0.1   # (m/s, rad)

def main():
    spin(Algo)
```

Included: **follow_the_gap** (reactive: bubbles out the closest obstacle and aims into the widest free gap) and **wall_follow** (PID on the distance to the left wall).

5. Moving to the real car

Not done yet. The car keeps its own ROS; the Mac only connects over SSH.

One-time setup

- Get from the car's current driver: its Wi-Fi IP (`hostname -I` on the car), the password for user `swat-roboracer-2026`, its ROS version (`ls /opt/ros`), and its workspace folder (usually ~/f1tenth_ws).
- Put the IP in `car.env` (`CAR_HOST`). Mac and car on the same Wi-Fi; `rr status` should say reachable.
- `rr car-key` once (copies an SSH key, so no more passwords), then `rr car` to check you get a shell.

Each session

- Battery charged. **Wheels off the ground** (car on a box) for the first run of any new algorithm.
- Terminal 1: `rr car-bringup` (lidar, motor controller, joystick). Leave it running.
- Terminal 2: `rr car-push` (copies and builds your algorithms on the car).
- Terminal 2: `rr car-run follow_the_gap -p max_speed:=1.0`.
- The stock F1TENTH stack only obeys the algorithm while you **hold the joystick's autonomous button** (usually RB). Let go and the car stops. Ctrl-C also stops it.
- Once it behaves on the box, run it on the floor in open space and raise max_speed gradually.

ForzaETH on the real car is a bigger project (their Docker on the car, mapping your track, tuning a steering table for your car). Get your own algorithms working on the car first.

6. Removing everything

```
rr down
```

```
orbctl delete forzaeth roboracer
brew uninstall --cask orbstack
rm /opt/homebrew/bin/rr
rm -rf ~/Documents/Projects/robotics/roboracer
```

Sources

Project	Link	Version used
f1tenth_gym_ros	github.com/f1tenth/f1tenth_gym_ros	dev-humble @ 0839576
f1tenth_gym	github.com/f1tenth/f1tenth_gym	dev-humble @ bdaec14
f1tenth_racetracks	github.com/f1tenth/f1tenth_racetracks	master
ForzaETH race_stack	github.com/ForzaETH/race_stack	ros2-humble @ 3f8eb3f
OrbStack	orbstack.dev	2.2.3
ROS 2 Humble	docs.ros.org/en/humble	apt, Ubuntu 22.04 arm64